- You can also use services from the DI container to configure an `IOptions<T>` object by creating a separate class that implements `IConfigureOptions<T>`. This class can use DI in the constructor and is used to lazily build a requested `IOptions<T>` object at runtime.
- You can replace the built-in DI container with a third-party container. Third-party containers often provide additional features, such as convention-based dependency registration, assembly scanning, and property injection.

---

[1] Although the promotion of DI as a core practice has been applauded, this abstraction has seen some controversy. This post, titled "What's wrong with the ASP.NET Core DI abstraction?", from one of the maintainers of the SimpleInjector DI library, describes many of the arguments and concerns: http://mng.bz/yYAd. You can also read more about the decisions at http://mng.bz/6DnA.